MiniShop API — Contract Tests & Review Notes

1. What was built

`api-tests.postman_collection.json` is a Postman v2.1 collection with **6 requests** covering all 3 endpoints. Each request has its contract payload saved as an **Example** and a **Tests** script written with `pm.test()`.

Request	Saved example	What the tests assert
`GET /products`	200 product list	Status 200. Non-empty array. Every product has `id`, `name`, `price` and `inStock`. Types: integer, non-empty string, number, boolean. Passes a JSON schema. Unique ids. Prices > 0 with at most 2 decimals. Catalogue equals the contract data and the MiniShop v1 UI
`POST /orders`	201 Created	Status 201. Required fields present. `orderId` is a string in `ORD-<n>` format. `status` is the string `"confirmed"`. `total` is a positive number with at most 2 decimals and **equals Σ price × quantity** (worked out in integer cents). `estimatedDelivery` is a real `YYYY-MM-DD` date (rejects rollovers such as `2026-02-30`). No `error` key. Saves `orderId` for the next request
`POST /orders` (no `customerEmail`)	400 Bad Request	Precondition: the body really omits `customerEmail`. Status 400. `error` is a non-empty string equal to `customerEmail is required`. No order fields in the body
`POST /orders` (productId 3)	409 Conflict	Precondition: product 3 is `inStock: false` in the catalogue. Status 409. `error` names the product (`USB-C Hub is currently out of stock`). No order fields
`GET /orders/{{orderId}}`	200 order found	Status 200. Required fields and types (`items` is an array of integer `productId` and positive integer `quantity`). The returned `orderId` matches the URL. `total` equals Σ price × quantity. **Consistent with the POST response** (`orderId`, `status`, `total`, `items`)
`GET /orders/ORD-9999`	404 Not Found	Status 404. `error` equals `Order not found`. No order data leaked

There are also 2 collection-level tests on every response: `Content-Type` is JSON, and the body parses as JSON. That makes **49 assertions** in total.

How the tests run without a server. Every request sends an `x-mock-response-name` header naming its example, so it works the same way against:

- a **Postman mock server**: Collection → ⋯ → *Mock collection*, then set `baseUrl` to the mock URL and click *Run*; or
- the **local mock** in `mock-server/` (next to the collection). This is a zero-dependency Node server that reads the saved examples straight from the collection file, so there's only one copy of the payloads. `npm test` there starts it and runs the collection with Newman.

Against a real backend, point `baseUrl` at it. The header is simply ignored.

Proof the assertions have teeth. I ran a mutation check: I broke the examples (total `39.98` → `39.99`, `orderId` as a number, `estimatedDelivery: 2026-02-30`, 400 → 422, a price as a string, the 404 error key renamed) and re-ran. 12 assertions failed, each with a message naming the broken field. Then I restored the examples. The suite is 49/49 green.

2. Notes for the API developer

Ranked by impact. Each note says what the tests can't currently check because the contract is silent.

High

1. **No authentication, and order IDs are sequential.** `GET /orders/ORD-1042` needs no credentials, and IDs look incremental, so anyone can walk `ORD-1041`, `ORD-1043`, and so on, and read other customers' orders (an IDOR). This

gets worse as soon as orders carry PII such as `customerEmail` or an address. **Ask:** define authentication for `/orders*`, enforce ownership checks, and use non-guessable IDs (UUID or ULID).

2. **Money is a JSON float with no currency.** `19.99` and `39.98` as binary floats invite rounding bugs in every client; the MiniShop UI already sums floats. **Ask:** use integer minor units (`"totalCents": 3998`) or decimal strings, and add `"currency": "USD"`.
3. **Error bodies are just a human sentence.** `{ "error": "USB-C Hub is currently out of stock" }` gives clients no machine-readable code, no field and no product id. To know *which* item failed, a client has to parse English, and that breaks the moment the copy changes or gets translated. **Ask:** one error envelope for every 4xx, for example RFC 9457 `application/problem+json`: `{ "type", "title", "status", "detail", "code": "OUT_OF_STOCK", "productIds": [3] }` or `"field": "customerEmail"`. Return all validation errors at once, not just the first.

Medium

4. **The order looks different in POST and GET.** The 201 response has `estimatedDelivery` but no `items`. The GET response has `items` but no `estimatedDelivery`. Neither has `customerEmail`, `createdAt`, unit prices or line totals. **Ask:** one `Order` schema returned by both, so the tests can assert POST and GET are equal as a whole.
5. **Naming is inconsistent.** Products use `id`, orders use `orderId`, and line items use `productId`. **Ask:** pick one convention, such as `id` on every resource and `<resource>Id` only for references.
6. **Validation rules are undefined**, so these inputs have no expected result to test against: `items: []`, `quantity` of `0`, `-1`, `1.5` or `1e9`, duplicate `productId` lines, an unknown `productId` (404, 400 or 422?), a malformed email, unknown extra fields, and a malformed id in `GET /orders/abc` (400 or 404?). **Ask:** document the status and error `code` for each.
7. **The 409 semantics are thin.** What happens when several items are out of stock? Is a partial order ever accepted? Stock is a boolean only, so "quantity 5 but only 2 left" can't even be expressed. **Ask:** list every offending product in the 409, and consider `stockQuantity`. Also document why this is 409 rather than 422, and apply it consistently.
8. **Enums and formats aren't specified.** `status` has one sample value (`confirmed`), but clients need the full set (pending, shipped, cancelled?). The format and time zone of `estimatedDelivery` are implied, not stated, and without `createdAt` there's nothing to check it against. **Ask:** publish the enum, and state ISO 8601 date plus time-zone rules.

Low

9. **No `Location` header** is specified for `201 Created` (it should be `/orders/ORD-1042`).
10. **No idempotency on `POST /orders`.** A retry or double-click creates duplicate orders. **Ask:** support an `Idempotency-Key` header.
11. **No versioning** (`/v1`), and no pagination or filtering for `/products`. `Content-Type` and `Accept` expectations aren't documented.
12. **A cross-layer gap to share with the UI team:** the API reports USB-C Hub as `inStock: false`, but MiniShop v1 lets users add it and check out (test plan SG-08). Once wired up, users would first learn it's unavailable from a 409 at checkout.